
1 P1

We can think of reverse-mode automatic differentiation as a generalization of the error backpropagation procedure.

As with forward mode, we augment each intermediate variable v_i with additional variables, in this case called *adjoint variables*, denoted $\bar{v}_i$.

Consider again a situation with a single output function f for which the adjoint variables are defined by

$$\bar{v}_i = \frac{\partial f}{\partial v_i}. \quad (8.68)$$

These can be evaluated sequentially starting with the output and working backwards by using the chain rule of calculus:

$$\bar{v}_i = \frac{\partial f}{\partial v_i} = \sum_{j \in \text{ch}(i)} \frac{\partial f}{\partial v_j} \frac{\partial v_j}{\partial v_i} = \sum_{j \in \text{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}. \quad (8.69)$$

Here $\text{ch}(i)$ denotes the *children* of node i in the evaluation trace graph, in other words the set of nodes that have arrows pointing into them from node i .

The successive evaluation of the adjoint variables represents a flow of information backwards through the graph, as we saw previously.

2 P2

If we again consider the specific example function given by (8.50) to (8.56), we obtain the following evaluation equations for the evaluation of the adjoint variables

$$\bar{v}_7 = 1 \tag{8.70}$$

$$\bar{v}_6 = \bar{v}_7 \tag{8.71}$$

$$\bar{v}_5 = \bar{v}_7 \tag{8.72}$$

$$\bar{v}_4 = -\bar{v}_6 \tag{8.73}$$

$$\bar{v}_3 = \bar{v}_5 v_5 + \bar{v}_6 \tag{8.74}$$

$$\bar{v}_2 = \bar{v}_3 v_1 + \bar{v}_4 \cos(v_2) \tag{8.75}$$

$$\bar{v}_1 = \bar{v}_3 v_2. \tag{8.76}$$

Note that these start at the output and then flow backwards through the graph to the inputs.

Even with multiple inputs, only a single backward pass is required to evaluate the derivatives.

For a neural network error function, the derivatives of E with respect to the weight and biases are obtained as the corresponding adjoint variables.

However, if we now have more than one output then we need to run a separate backward pass for each output variable.

3 P3

Reverse mode is often more memory intensive than forward mode because all of the intermediate primal variables must be stored so that they will be available as needed when evaluating the adjoint variables during the backward pass.

By contrast, with forward mode, the primal and tangent variables are computed together during the forward pass, and therefore variables can be discarded once they have been used.

It is therefore also generally easier to implement forward mode compared to reverse mode.

4 P4

For both forward-mode and reverse-mode automatic differentiation, a single pass through the network is guaranteed to take no more than 6 times the computational cost of a single function evaluation.

In practice, the overhead is typically closer to a factor of 2 or 3 (Griewank and Walther, 2008).

Hybrids of forward and reverse modes are also of interest.

One situation in which this arises is in the evaluation of the product of a Hessian matrix with a vector, which can be calculated without explicit evaluation of the full Hessian (Pearlmutter, 1994).

Here we can use reverse mode to calculate the gradient of code, which itself has been generated by the forward mode.

We start from a vector $\mathbf{b}$ and a point $\mathbf{x}$ at which the Hessian-vector product is to be evaluated.

By setting $\dot{\mathbf{x}} = \mathbf{v}$ and using forward mode, we obtain the directional derivative $\mathbf{v}^\top \nabla f$.

This is then differentiated using reverse mode to obtain $\nabla^2 f \mathbf{v} = \mathbf{H} \mathbf{v}$.

If W is the number of parameters in the neural network then this evaluation has $\mathcal{O}(W)$ complexity even though the Hessian is of size $W \times W$.

The Hessian itself can also be evaluated explicitly using automatic differentiation but this has $\mathcal{O}(W^2)$ complexity.

翻译:

对于前向模式和反向模式自动微分，单次通过网络的计算开销均保证不超过单次函数求值计算代价的 6 倍。

在实践中，这一额外开销通常更接近 2 到 3 倍（Griewank and Walther, 2008）。

前向与反向模式的混合方法同样值得关注。

一个典型应用场景是 Hessian 矩阵与向量的乘积计算，该运算无需显式构造完整的 Hessian 矩阵即可完成（Pearlmutter, 1994）。

具体而言，我们可以对由前向模式生成的代码再应用反向模式来计算其梯度。

设待计算 Hessian-向量积的点为 $\mathbf{x}$ ，方向向量为 $\mathbf{v}$ 。

在前向模式中令 $\dot{\mathbf{x}} = \mathbf{v}$ ，即可得到方向导数 $\mathbf{v}^\top \nabla f$ ；随后对该结果施加反向模式微分，便得到 $\nabla^2 f \mathbf{v} = \mathbf{H}\mathbf{v}$ 。

若神经网络参数数量为 W ，尽管 Hessian 矩阵本身大小为 $W \times W$ ，上述 Hessian-向量积的计算复杂度仅为 $\mathcal{O}(W)$ 。

当然，也可借助自动微分直接显式计算完整 Hessian 矩阵，但其复杂度将升至 $\mathcal{O}(W^2)$ 。

重点难点解读：

计算复杂度的常数上界：原文提到的“6 倍”是自动微分理论中的经典上界（源自 Griewank 等人的工作），表明无论计算图多复杂，AD 的单次遍历开销相对于原函数求值仅有常数量级的增长。这从理论上保证了 AD 在大规模模型训练中的可行性，而实际中 2-3 倍的开销则反映了现代编译器优化与算子融合的效果。

Hessian-向量积的核心价值：在深度学习优化（如二阶方法、自然梯度）和不确定性估计中，往往只需要 $\mathbf{H}\mathbf{v}$ 而非完整 $\mathbf{H}$ 。Pearlmutter (1994) 提出的“前向 + 反向”混合策略巧妙地将 $\mathcal{O}(W^2)$ 问题降为 $\mathcal{O}(W)$ ，这是当前大模型二阶近似算法（如 K-FAC、LiSSA 等）的理论基石。

“对代码求梯度”的理解难点：文中“calculate the gradient of code, which itself has been generated by the forward mode”表述较为抽象。其本质是指：前向模式输出的不仅是数值 $\mathbf{v}^\top \nabla f$ ，更是一段可微的计算过程（或计算图）；反向模式作用于该中间计算过程，等价于对复合函数 $f(\mathbf{x} + t\mathbf{v})$ 关于 t 再求一次导数，从而提取出二阶信息。理解这一点需要同时掌握 AD 的程序变换视角与链式法则的数学视角。

$\mathcal{O}(W)$ 与 $\mathcal{O}(W^2)$ 的实践鸿沟：当 W 达到百万乃至十亿量级时， $\mathcal{O}(W^2)$ 的存储与计算完全不可行，而 $\mathcal{O}(W)$ 的 HVP 使得二阶信息在大模型时代仍具实用价值。这也解释了为何现代深度学习框架普遍原生支持 `grad(grad(...))` 或专用 HVP 接口，而非直接暴露 Hessian 矩阵。